

Señales aleatorias

Tema 6 · Parte 1: variables aleatorias, muestreo y análisis

Jesús Chacón Sombría

Departamento de Arquitectura de Computadores y Automática
Universidad Complutense de Madrid

Grado en Ingeniería Electrónica de Comunicaciones
Curso 2026-2027

Objetivos del tema

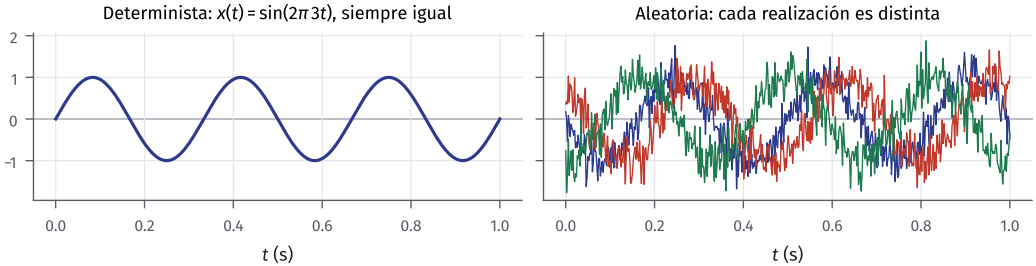
- Describir lo que no se puede predecir: ruido, interferencias, mensajes, medidas.
- Variables aleatorias: funciones de probabilidad, media y varianza; generar muestras con el ordenador y estimar a partir de ellas.
- Varias variables: distribución conjunta, independencia, covarianza y correlación.
- Señales aleatorias: estacionariedad, ergodicidad, autocorrelación y correlación cruzada.
- Análisis espectral: densidad espectral de potencia y respuesta de un sistema LTI a una señal aleatoria.

Partes: 1 Variables aleatorias · 2 Varias variables · 3 Señales aleatorias · 4 Análisis espectral.

Contenido

1. Señales deterministas y aleatorias
2. Variable aleatoria
3. Distribuciones habituales
4. Muestreo de variables aleatorias
5. Análisis de muestras

¿Por qué señales aleatorias?



- Una señal determinista se conoce exactamente en todo instante: una fórmula, una tabla.
- Una señal aleatoria (o estocástica) no se puede predecir: cada vez que se observa sale distinta. Ruido térmico, voz, datos transmitidos, vibraciones, medidas de un sensor.
- No se describe valor a valor, sino estadísticamente: qué valores son probables, cuánto varían, cómo se parecen los valores próximos.

Contenido

1. Señales deterministas y aleatorias
2. Variable aleatoria
3. Distribuciones habituales
4. Muestreo de variables aleatorias
5. Análisis de muestras

Variable aleatoria

Una variable aleatoria X asigna un número a cada resultado de un experimento aleatorio. Antes del experimento solo se sabe con qué probabilidad tomará cada valor; el valor obtenido, x, es una realización o muestra.

- Discreta: toma valores sueltos x_i (la cara de un dado, un bit recibido). Se describe con la función de probabilidad $p_X(x_i) = P(X = x_i)$, con $\sum_i p_X(x_i) = 1$.
- Continua: toma valores en un intervalo (una tensión de ruido). Se describe con la función de densidad $f_X(x) \geq 0$:

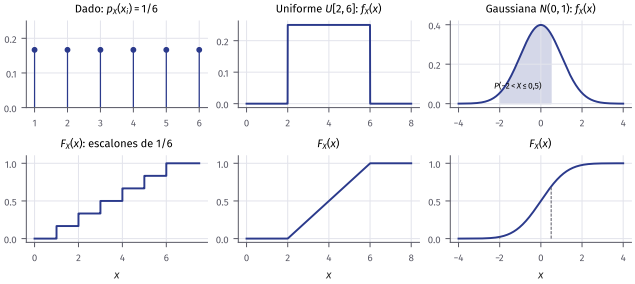
$$P(a < X \leq b) = \int_a^b f_X(x)\,\mathrm{d}x, \qquad \int_{-\infty}^{\infty} f_X(x)\,\mathrm{d}x = 1.$$

$f_X(x)$ no es una probabilidad: puede ser mayor que 1, y $P(X = x) = 0$.

Función de distribución

$$F_X(x) = P(X \le x)$$

- Vale para discretas y continuas.
- Crece de $F_X(-\infty) = 0$ a $F_X(\infty) = 1$.
- $P(a < X \le b) = F_X(b) - F_X(a)$.
- Continuas: $f_X(x) = \frac{\mathrm{d}F_X}{\mathrm{d}x}$.
- Discretas: escalones de altura $p_X(x_i)$ en cada x_i.



Esperanza, media y varianza

Esperanza

$$E\{g(X)\} = \sum_i g(x_i)\, p_X(x_i) \qquad \text{o} \qquad E\{g(X)\} = \int_{-\infty}^{\infty} g(x)\, f_X(x)\, \mathrm{d}x$$

- Media: $\mu_X = E\{X\}$, el centro de gravedad de la distribución.
- Valor cuadrático medio: $E\{X^2\}$. Si X es una tensión sobre $1\,\Omega$, es la potencia media.
- Varianza: $\sigma_X^2 = E\{(X - \mu_X)^2\} = E\{X^2\} - \mu_X^2$. Su raíz, σ_X, es la desviación típica.
- La esperanza es lineal: $E\{aX + b\} = a\mu_X + b$ y $\mathrm{Var}\{aX + b\} = a^2\sigma_X^2$.

Dado

$\mu_X = \frac{1+\cdots+6}{6} = 3{,}5$; $E\{X^2\} = \frac{91}{6}$; $\sigma_X^2 = \frac{91}{6} - 3{,}5^2 = \frac{35}{12}$.

Contenido

1. Señales deterministas y aleatorias
2. Variable aleatoria
3. Distribuciones habituales
4. Muestreo de variables aleatorias
5. Análisis de muestras

Uniforme y gaussiana

Uniforme $U[a, b]$:

$$f_X(x) = \begin{cases} \frac{1}{b-a}, & a \le x \le b \\ 0, & \text{resto} \end{cases}$$

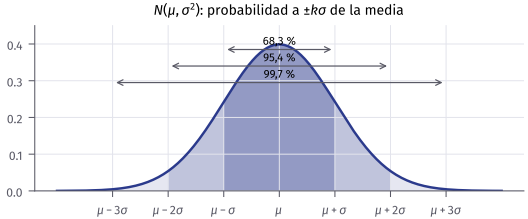
$\mu_X = \frac{a+b}{2}$, $\sigma_X^2 = \frac{(b-a)^2}{12}$, $E\{X^2\} = \frac{a^2+ab+b^2}{3}$.

Error de cuantificación, fase de una portadora, números aleatorios del ordenador.

Gaussiana o normal $N(\mu, \sigma^2)$:

$$f_X(x) = \frac{1}{\sqrt{2\pi}\,\sigma} e^{-\frac{(x-\mu)^2}{2\sigma^2}}$$

Ruido térmico y, en general, la suma de muchas causas pequeñas e independientes (parte 2).



- $Z = (X - \mu)/\sigma$ es $N(0, 1)$; F_Z se tabula como $\Phi(z)$.
- $aX + b$ es $N(a\mu + b,\ a^2\sigma^2)$.

Contenido

1. Señales deterministas y aleatorias
2. Variable aleatoria
3. Distribuciones habituales
4. Muestreo de variables aleatorias
5. Análisis de muestras

Generar muestras con el ordenador

- Los generadores de números pseudoaleatorios dan muestras de $U[0,1)$: `rand` en Matlab, `rng.random` en Python. A partir de ellas se obtiene cualquier distribución.
- Simular un sistema con entradas aleatorias muchas veces permite estimar lo que es difícil de calcular: es el método de Monte Carlo.
- Métodos:
  - variables discretas: método de la ruleta;
  - inversa de la función de distribución;
  - Box-Muller para gaussianas (Matlab y Python traen `randn` y `rng.standard_normal`);
  - aceptación y rechazo, cuando no se puede invertir F_X;
  - mezclas.

Fijar la semilla (`rng(0)` en Matlab, `np.random.default_rng(0)` en Python) hace los resultados reproducibles.

Variables discretas: método de la ruleta

- Se reparte $[0, 1)$ en tramos de longitud $p_X(x_i)$, como los sectores de una ruleta.
- Se genera $u \sim U[0, 1)$ y se devuelve el x_i cuyo tramo contiene a u:

$$x = x_i \quad \text{si} \quad F_X(x_{i-1}) \le u < F_X(x_i).$$

- El tramo i mide $p_X(x_i)$, así que sale con esa probabilidad.

Dado cargado

p = (0,1; 0,1; 0,1; 0,1; 0,1; 0,5),
F = (0,1; 0,2; ...; 0,5; 1). Con u = 0,37:
$F(3) = 0{,}3 \le 0{,}37 < 0{,}4 = F(4)$, sale un 4. Con u = 0,62 sale un 6.

Erratas antiguas: la condición de la ruleta debe incluir la igualdad en un extremo; si no, los u que caen justo en un borde no tienen valor asignado.

En código: método de la ruleta

Python

```python
import numpy as np

rng = np.random.default_rng(1)
p = np.array([.1, .1, .1, .1, .1, .5])
F = np.cumsum(p)                  # 0.1 ... 1
u = rng.random(60000)
x = np.searchsorted(F, u, 'right') + 1
frec = np.bincount(x, minlength=7)[1:]
print(frec/u.size)                # ~ p
print(x.mean(), np.sum(np.arange(1, 7)*p))
```

Matlab

```matlab
p = [.1 .1 .1 .1 .1 .5];          % dado
F = cumsum(p);                    % 0.1 ... 1
u = rand(1, 60000);
x = zeros(size(u));
for i = 1:numel(u)
  x(i) = find(u(i) < F, 1);  % 1er F > u
end
frec = accumarray(x', 1)'/numel(u);
disp(frec)                        % ~ p
disp([mean(x), sum((1:6).*p)])
```

Inversa de la función de distribución

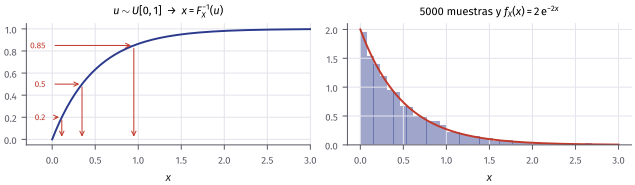
Resultado

Si $U \sim U[0,1]$ y F_X es continua y creciente, $X = F_X^{-1}(U)$ tiene distribución F_X.

Porque
$P(X \le x) = P(U \le F_X(x)) = F_X(x)$.

Exponencial: $F_X(x) = 1 - e^{-\lambda x}$, así que

$$x = -\frac{1}{\lambda} \ln(1-u).$$



El método de la ruleta es la versión discreta de este. Aplicado al revés, $F_X(X)$ es uniforme: es la ecualización del histograma (tema 5).

En código: exponencial e inversa



Python

```python
import numpy as np

rng = np.random.default_rng(0)
lam, N = 2.0, 100000
u = rng.random(N)                  # U[0,1)
x = -np.log(1 - u)/lam             # F^-1(u)
print(x.mean(), 1/lam)             # media
print(x.var(ddof=1), 1/lam**2)     # varianza
print(np.mean(x > 1), np.exp(-lam))
```

Matlab

```matlab
lam = 2; N = 100000;
u = rand(1, N);                    % U[0,1)
x = -log(1 - u)/lam;               % F^-1(u)
disp([mean(x), 1/lam])             % media
disp([var(x), 1/lam^2])            % varianza
disp([mean(x > 1), exp(-lam)])
```

`var` de Matlab y `var(ddof=1)` de numpy dividen por N – 1; `np.var` sin más, por N.

Gaussianas: Box-Muller

- F de la gaussiana no tiene inversa explícita.
- Truco: un par (Z_1, Z_2) de gaussianas independientes, en polares, tiene ángulo uniforme y radio con $F_R(r) = 1 - e^{-r^2/2}$, que sí se invierte.
- Con $u_1, u_2 \sim U[0,1)$ independientes:

$$r = \sqrt{-2\ln(1-u_1)}, \qquad \theta = 2\pi u_2,$$

$$z_1 = r\cos\theta, \qquad z_2 = r\sin\theta.$$

- Se obtiene un par de muestras $N(0,1)$ independientes. Para $N(\mu, \sigma^2)$: $x = \mu + \sigma z$.

Una pareja

$u_1 = 0{,}6$, $u_2 = 0{,}1$: $r = \sqrt{-2\ln 0{,}4} = 1{,}354$, $\theta = 0{,}2\pi$; $z_1 = 1{,}095$ y $z_2 = 0{,}796$.

La versión antigua decía «se genera un par...» sin terminar la frase.

En código: Box-Muller



Python

```python
import numpy as np

rng = np.random.default_rng(2)
N = 50000
u1, u2 = rng.random(N), rng.random(N)
r = np.sqrt(-2*np.log(1 - u1))   # radio
th = 2*np.pi*u2                  # ángulo
z1, z2 = r*np.cos(th), r*np.sin(th)
x = 5 + 2*z1                     # N(5, 4)
print(z1.mean(), z1.std(), z2.std())
print(np.corrcoef(z1, z2)[0, 1]) # ~ 0
print(x.mean(), x.var())         # ~ 5, 4
print(np.mean(abs(z1) < 1))      # ~ 0.683
```

Matlab

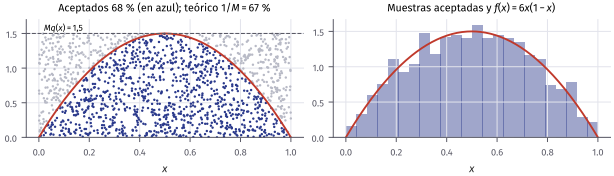
```matlab
N = 50000;
u1 = rand(1, N); u2 = rand(1, N);
r = sqrt(-2*log(1 - u1));        % radio
th = 2*pi*u2;                    % ángulo
z1 = r.*cos(th); z2 = r.*sin(th);
x = 5 + 2*z1;                    % N(5, 4)
disp([mean(z1), std(z1), std(z2)])
c = corrcoef(z1, z2); disp(c(1, 2))  % ~ 0
disp([mean(x), var(x)])          % ~ 5, 4
disp(mean(abs(z1) < 1))          % ~ 0.683
```

Aceptación y rechazo

- Se busca una densidad $q(x)$ fácil de muestrear y un M con

$$Mq(x) \geq f(x) \quad \text{para todo } x.$$

- Se genera $x \sim q$ y $u \sim U[0,1)$; se acepta x si $u\,Mq(x) \leq f(x)$.
- Los puntos $(x, uMq(x))$ caen uniformes bajo Mq; los aceptados, uniformes bajo f.
- Se acepta una fracción $1/M$: conviene Mq ajustada a f.

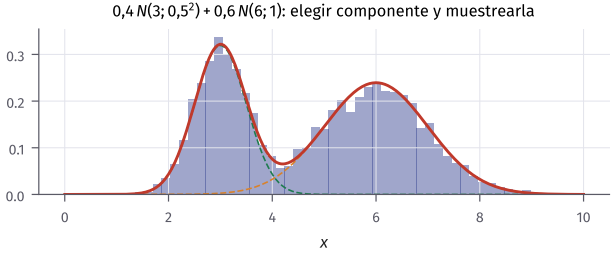


Errata antigua: la condición es $Mq(x) \geq f(x)$, con igualdad permitida, y no $Mq(x) - f(x) > 0$.

Mezclas

$$f_X(x) = \sum_{k=1}^{K} \pi_k \, f_k(x), \qquad \sum_k \pi_k = 1$$

- Modela poblaciones con varios grupos (dos tipos de pieza, ruido con dos regímenes).
- Muestreo en dos pasos: elegir la componente k con probabilidad π_k (ruleta) y muestrear f_k.

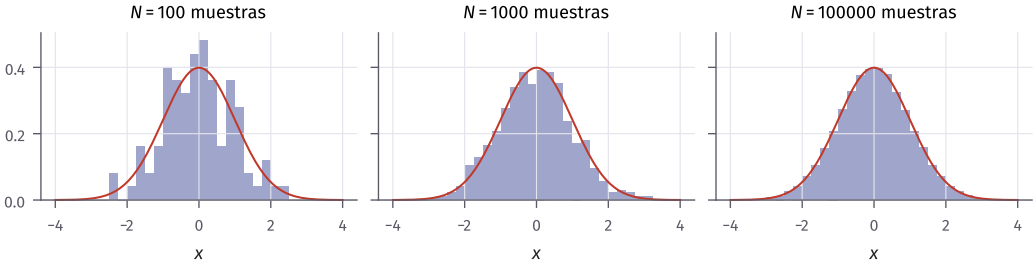


La transparencia antigua dejaba la mezcla sin terminar: «$p_X(x) = 0{,}4N(x, 3, 0{,}5)+$».

Contenido

1. Señales deterministas y aleatorias
2. Variable aleatoria
3. Distribuciones habituales
4. Muestreo de variables aleatorias
5. Análisis de muestras

Histograma



- Se divide el eje en intervalos de anchura Δ y se cuentan las muestras n_j de cada uno.
- $\hat{f}_X(x) = \frac{n_j}{N\Delta}$ en el intervalo j estima la densidad (el área total es 1). Más muestras, mejor estimación; intervalos más estrechos, más detalle pero más ruido.

Matlab: `histogram(x, 'Normalization', 'pdf');` *Python:* `plt.hist(x, density=True).`

Estimadores

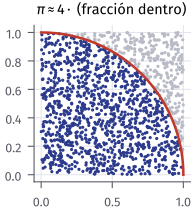
Con N muestras independientes $x_1, \ldots, x_N$:

$$\hat{\mu} = \frac{1}{N}\sum_{i=1}^{N} x_i, \qquad \hat{\sigma}^2 = \frac{1}{N-1}\sum_{i=1}^{N}(x_i - \hat{\mu})^2, \qquad E\{\hat{g(X)}\} = \frac{1}{N}\sum_{i=1}^{N} g(x_i).$$

- Son también variables aleatorias. $\hat{\mu}$ es insesgado ($E\{\hat{\mu}\} = \mu$) y su varianza es σ^2/N: el error típico baja como $1/\sqrt{N}$.
- Con $N - 1$ el estimador de la varianza es insesgado; con N (el de máxima verosimilitud para la gaussiana) se queda corto en un factor $(N-1)/N$.
- Monte Carlo: $E\{g(X)\} = \int g(x) f_X(x)\,dx$ se aproxima por la media de g sobre las muestras, aunque la integral no tenga solución cerrada.

Erratas antiguas: la suma de Monte Carlo usaba el índice n y escribía x_i, y el estimador de la varianza dividía por N sin advertir el sesgo.

Monte Carlo





Python

```python
import numpy as np

rng = np.random.default_rng(4)
for N in (100, 10000, 1000000):
    x = rng.standard_normal(N)  # X ~ N(0,1)
    g = np.cos(x)               # E{cos X}?
    est = g.mean()
    err = g.std(ddof=1)/np.sqrt(N)
    print(N, est, err)
print(np.exp(-0.5))             # exacto
```

Matlab

```matlab
for N = [100 10000 1000000]
  x = randn(1, N);              % X ~ N(0,1)
  g = cos(x);                    % E{cos X}?
  est = mean(g);
  err = std(g)/sqrt(N);
  disp([N, est, err])
end
disp(exp(-0.5))                  % exacto
```

Conceptos importantes

- Variable aleatoria: discreta (p_X) o continua (f_X); $F_X(x) = P(X \le x)$.
- $E\{\cdot\}$ lineal; μ_X, $E\{X^2\}$, $\sigma_X^2 = E\{X^2\} - \mu_X^2$.
- Uniforme y gaussiana; $Z = (X - \mu)/\sigma$.
- Muestreo: ruleta, inversa, Box-Muller, rechazo, mezclas.
- Histograma normalizado $n_j/(N\Delta)$.
- Estimadores: media, varianza con $N - 1$; error $\propto 1/\sqrt{N}$.

Apuntes del Tema 6, sección 1 · Hoja de ejercicios, ejercicios 1–5.